

Config-Driven White-Label Theming

One app and one design system. Each brand is a JSON file, with no widget changes.

Flutter

Dart

dart_mappable

InheritedWidget

CI contract tests

Widgetbook

The problem

A food-ordering product had to ship as several restaurant brands from a single codebase. Each brand needs its own colours, fonts, corner radii, button style, icons and identity strings, but the business logic, screens and components should stay shared and never fork.

1

shared app + component library

0

widget changes per new brand

8

typed + free-form token sections

CI

every brand validated on each PR

What I built

A schema-driven theming layer. Each brand is a thin client package that owns one `theme.json`. At start-up the JSON is validated, resolved and turned into a typed theme object that every component reads from an `InheritedWidget`.

Brand package
`theme.json`

→

Load
asset bundle

→

Resolve
{palette.x}
refs

→

Parse
typed
DsTheme

→

Provide
DsProvider

→

Components
re-skinned

1 · Brand = a package + a JSON file

Monorepo layout: a shared `design_system` (components, theme model, validators), a shared `foodtech` app, and one small shell package per brand holding only brand-specific assets (theme, fonts, images, Firebase config, splash). Brands never depend on each other.

Shared	Per brand
~40 reusable <code>Ds*</code> components, screens, state management, repositories, theme engine	<code>theme.json</code> , fonts, imagery, app icons & splash, entry point, theme contract test

2 · A schema that scales

Strongly-typed groups (`brandColor`, `color`, `radius`, `spacing`, `typography`, `font`) carry the core design language. A free-form `components` map carries per-component styling: button variants, shadows, durations, curves, icon roles and brand identity, so new components can become themable without changing the schema.

```
"palette": { "primary": "#E12726ff" },
"color": { "button": { "primary": "{palette.primary}" } },
"components": { "button": { "variants": { "primary": {
  "background": "{palette.primary}", "background-pressed": "#e5504fff" } } } }
```

3 · Single source of truth with references

A `{palette.x}` reference resolver lets a colour be defined once and reused everywhere, including chained references. Changing a brand colour is a one-line edit. Unknown paths and reference cycles throw at start-up, so mistakes surface immediately instead of as a wrong colour in production.

4 · Typed, safe accessors with graceful fallback

```
theme.color.button.primary // typed group
theme.componentColor ('button.variants.primary.background', fallback)
theme.componentDouble ('button.height.large', 56)
theme.componentDuration('toggle.animation-duration-ms', 200ms)
theme.componentCurve ('toggle.curve', Curves.easeInOut)
theme.componentIcon ('icons.home', fallbackGlyph)
```

Every accessor requires a fallback equal to the previous hard-coded value. That let me tokenise the component library **incrementally with zero visual regressions**, and means partial brand files never crash a screen. Colours use CSS-style `RRGGBBAA` hex, parsed by a dedicated mapper.

Themable icons: Flutter's `IconData` can't be created from a string, so JSON stores semantic roles (`home` , `scan` , `account`) that map to names in an icon registry; each brand picks its own glyph set.

5 · Runtime wiring

```
// brand's main()
final json = jsonDecode(
  await rootBundle.loadString(
    'assets/theme.json'));
runApp(FoodTechApp(data: json));
```

```
// in the shared app
DsProvider(
  theme: DsTheme.fromData(json),
  child: ...)

// in any component
DsThemeProvider.watch(context).theme
```

Even non-visual brand configuration (app name, support e-mail, payment merchant label) is read from the same JSON through a small `ClientConfig` , so a brand is described in exactly one place. Each brand ships only its own theme and fonts, since the design system itself bundles none.

6 · Quality gates for every brand

- **Fail-fast parsing:** missing sections/keys and broken references stop the app at boot.
- **ThemeValidator:** runs on the raw JSON and collects *all* violations (hex shape, icon names, curve ids, references) in one pass, suitable for a pre-build CI gate.
- **Conformance report:** compares a brand against a canonical fixture for key parity, kebab-case naming, required icon roles and identity tokens.
- **Per-brand contract tests** run in CI: brand colours, button variants, icon resolution, client config, zero validator errors, and a check that **every font family used in the JSON is actually bundled** in the brand's pubspec.

7 · Previewing brands

A Widgetbook gallery has a theme-picker addon that loads each brand's `theme.json` and swaps the active theme through a signal, so designers can compare any component across brands in the browser.

Outcome

- **New brand = new JSON + shell package:** no component or screen code changes.
- **One codebase, no forks:** all features and fixes reach every brand at once.
- **Safe by construction:** schema, validator and CI tests catch bad themes before release.
- **Lean binaries:** each app contains only its own brand's tokens and fonts.
- **Incremental and regression-free:** fallbacks allowed gradual migration of hard-coded styling to tokens.

Skills demonstrated

Theming architecture · Schema design & validation · Monorepo / multi-package Flutter · `dart_mappable` serialization · InheritedWidget state propagation · CI contract testing · Widgetbook tooling